ALOHAR MiniOS

Technical Documentation

INTRODUCTION

Alohar MiniOS is a lightweight, command-line simulated operating system developed entirely in Python. Created as a final project for IT3131, the program demonstrates how an operating system manages core tasks behind the scenes without requiring actual hardware-level changes. Through a terminal interface, users can securely log in, navigate the host machine's directories, start and terminate simulated background tasks, and allocate virtual system memory.

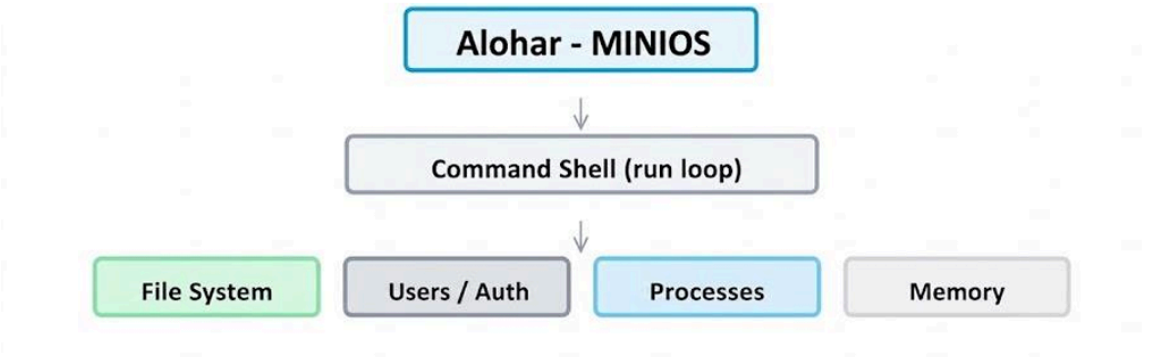
PROJECT OBJECTIVES

The main objectives of developing AloharOS are:

- To understand the core concepts of an operating system by building a working simulation using object-oriented Python.
- To implement a functional command-line interface (CLI) that continuously reads user input, sanitizes it, and routes it to the appropriate functions.
- To simulate RAM allocation and task management logic safely within an isolated environment.
- To interact with the actual host file system using standard Python OS libraries while preventing program crashes through proper error handling.

SYSTEM ARCHITECTURE

Alohar MiniOS follows a monolithic architecture built around a single primary Python class named alohar. The central command shell serves as the router between the user and the core system functions.



When a user enters a command, the Command Shell processes the input, references a command dictionary, and triggers the appropriate subsystem.

The system consists of the following components:

- **Command Shell:** Processes user commands and routes them to the appropriate methods.
- **File System:** Handles directory navigation and file viewing through the pwd, ls, cd, and cat commands.
- **Process Management:** Simulates running tasks through the run, kill, and process commands.
- **Memory Management:** Simulates memory allocation and release through the allocate, free, and memory commands.
- **User and System Information:** Manages authentication, user sessions, and system information through the login, logout, whoami, systeminfo, and uptime commands.

PROGRAM STRUCTURE

To maintain a unified and easily executable program, the entire system is structured inside a single Python script named Main.py.

Main.py

The Main.py file serves as the program's entry point. It initializes system variables, including the boot time, memory limits, and built-in user accounts, and starts the command-line interface loop.

The run() Loop

The run() method contains a while self.is_running: loop that acts as the heartbeat of the operating system. It continuously waits for user input, separates the command from its arguments, and routes the request to the appropriate function.

Command Dictionary (self.cmd)

Instead of relying on numerous if-else statements, Alohar - MINIOS uses a dictionary to connect command strings, such as help, directly to their corresponding Python methods. This approach makes command routing more organized, readable, and easier to maintain.

FILE MANAGEMENT

File management connects the simulated operating system to the actual host computer's file system using Python's standard os library.

Directory Navigation

- pwd: Uses os.getcwd() to retrieve and display the current working directory.
- cd: Uses os.chdir() to change the current working directory.
- ls: Uses os.listdir('.') to retrieve and display the files and folders in the current directory.
- cat: Uses Python's open(file, 'r') function to read and display the contents of text files in the CLI.

Error Handling and Safety

File management commands are wrapped in try-except blocks to handle common errors, including FileNotFoundError, NotADirectoryError, and PermissionError. These exception handlers prevent the program from terminating unexpectedly when a user enters an invalid path, attempts to access an inaccessible file, or provides an incorrect directory.

PROCESS MANAGEMENT

Process management simulates how an operating system's Task Manager tracks running applications.

Process Storage

Alohar - MINIOS uses the self.processes dictionary to store simulated background tasks. Each process is assigned a unique Process ID (PID), which serves as the dictionary key.

Starting Processes

The run command starts a simulated process. The system assigns the current value of self.next_pid, records the process with a RUNNING status, and increments the PID counter to ensure that newly created processes receive different identifiers.

Terminating Processes

The kill command removes the specified process from the dictionary, simulating the termination of an application. The PID is not reused during the same session.

Displaying Processes

The process command iterates through the process dictionary and displays a formatted table of all active simulated tasks.

MEMORY MANAGEMENT

The memory management module simulates system RAM using basic arithmetic and integer variables.

Memory Capacity

The system starts with a predefined memory capacity of 4096 MB. The self.total_memory variable stores the total capacity, while self.used_memory tracks the amount of memory currently allocated. Initially, the used memory is set to 0 MB.

Memory Allocation

The allocate command converts the user's input into an integer and checks whether the requested amount exceeds the available memory. If sufficient memory is available and the request is valid, the

system adds the requested amount to self.used_memory. Otherwise, the allocation is rejected.

Memory Release

The free command releases previously allocated memory back into the available memory pool. The system checks whether the requested release exceeds the currently used memory. This prevents the used-memory counter from becoming negative.

USER MANAGEMENT

User management handles authentication and session tracking during system operation.

Authentication

Alohar MiniOS stores demonstration accounts, including admin and student, in a dictionary. Passwords are hashed using Python's hashlib.sha256() rather than being stored as plain text. The system uses hmac.compare_digest() to compare password hashes during authentication.

Session Locking

When the program starts, the run() loop calls the self.login() method. The user must provide a valid username and password before accessing the operating system's commands.

The getpass library hides password input in the terminal, preventing the password characters from being displayed while the user types.

User Tracking

After successful authentication, the username is stored in self.username. The system can retrieve and display the active username through the whoami and systeminfo commands.

ERROR HANDLING

Alohar MiniOS incorporates error-handling mechanisms to maintain stability and prevent common user-input mistakes from interrupting program execution.

Input Sanitization

User input is processed using .strip() to remove unnecessary leading and trailing spaces. The .lower() method is used to make command recognition case-insensitive.

Empty Input Handling

If the user presses Enter without entering a command, the program skips the current loop iteration. This prevents errors that could occur when attempting to process an empty input.

Unknown Commands

When a command does not exist in the self.cmd dictionary, the system displays a message indicating that the command is unknown. The CLI loop then continues running, allowing the user to enter another command.

Invalid Arguments

Commands that require additional arguments check whether the necessary parameters have been provided. For example, the cd command requires a directory path.

These checks prevent missing arguments from causing unexpected errors or interrupting program execution.

TESTING

The development team tested Alohar MiniOS using different input scenarios in the terminal to verify its functionality, error handling, and stability.

Authentication Testing

The team attempted to log in using incorrect passwords to verify that unauthorized access was rejected. Password input was also checked to ensure that characters were hidden while typing.

File System Testing

The team tested directory navigation and file reading by moving between directories and attempting to access nonexistent folders or read directories as files.

These tests verified that the appropriate exception handlers responded to invalid file system operations.

Memory and Process Boundary Testing

The team tested process termination by attempting to kill nonexistent PIDs. Memory management was tested by entering non-numeric values into the allocate command and attempting to allocate more than the available 4096 MB capacity.

These scenarios verified that invalid inputs and requests exceeding system limits were handled appropriately.

PROBLEMS ENCOUNTERED

During development, the team encountered several logic and structural challenges. These were addressed through debugging and improvements to the program's input handling and command routing.

Infinite Loops and Errors Caused by Missing Arguments

In early versions of the program, entering a command without the required arguments, such as typing cd without specifying a directory, could cause errors that interrupted program execution.

This issue was addressed by adding argument checks, such as if not args:, at the beginning of functions that require parameters. These checks ensure that the program handles incomplete commands safely.

Dictionary-Based Command Routing

Implementing the self.cmd dictionary required the program to pass the arguments list consistently to each command function.

Even commands that did not require additional arguments, such as clear or pwd, needed compatible function definitions to avoid parameter mismatches.

The team resolved this structural issue by ensuring that command methods followed a consistent parameter pattern, making the command-routing mechanism more reliable and maintainable.

CONCLUSION

Developing Alohar MiniOS provided the team with practical experience in understanding how operating systems use command shells to process user input, manage simulated memory, authenticate users, and interact with file systems.

The project also strengthened the team's understanding of object-oriented programming in Python, exception handling, dictionaries, and collaborative software development.

Through the implementation of its command-line interface and core subsystems, Alohar MiniOS demonstrates fundamental operating system concepts in a simplified and accessible environment. Overall, the project served as a valuable learning experience in applying programming concepts to the simulation of an operating system.